

7.2) What is the running time of insertion sort if all elements are equal?

Ans: -

The **running time** of insertion sort when all elements are equal is typically considered to be $O(n)$, where 'n' is the number of elements in the input array.

In **insertion sort**, when all elements are equal, the algorithm still needs to iterate through each element in the array to determine its correct position within the sorted portion.

On the other hand, if the input is presorted, the running time is $O(N)$, because the test in the inner for loop always fails immediately. Indeed, if the input is almost sorted insertion sort will run quickly. Because of this wide variation, it is worth analyzing the average-case behavior of this algorithm. It turns out that the average case is $O(N^2)$ for insertion sort, as well as for a variety of other sorting algorithms.

If all elements of the array are equal, then the Insertion Sort algorithm will consider this as the best-case scenario. In this case, the outer loop will still go through n (where n is the number of elements in the list) iterations. However, the inner loop will only perform one comparison per iteration, because each element is equal (thus already sorted in relation to each other).

However, since all **elements** are equal, the comparison and swapping steps are not required, resulting in a simpler and faster sorting process.

Therefore, the time complexity of insertion sort in this specific scenario is linear, as it only needs to traverse the array once to determine the final sorted order, regardless of the equal values.

So, for each of the n elements in the list, it makes a single comparison, which translates to a running time of $O(n)$. Thus, when all elements are equal, the time complexity of Insertion Sort is linear. This is quite efficient compared to worst-case scenario which has a quadratic time complexity of $O(n^2)$.

7.11) Show how heap sort processes the input 142, 543, 123, 65, 453, 879, 572, 434, 111, 242, 811, 102.

Ans: -

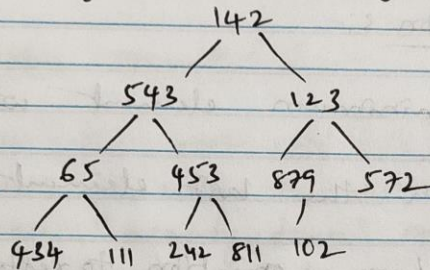
DSA - Week 5

7.11 show how heap sort processes the input 142, 543, 123, 65, 453, 879, 572, 434, 111, 242, 811, 102.

Ans:
Given list of inputs are as follows.

142	543	123	65	453	879	572	434	111	242	811	102
-----	-----	-----	----	-----	-----	-----	-----	-----	-----	-----	-----

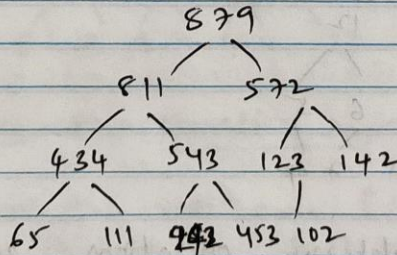
→ Representing the given array as a tree.



→ Here, we have created a tree from given array, as this is the unsorted tree, we have to create "build-maxheap".

~~142 543 123 65~~

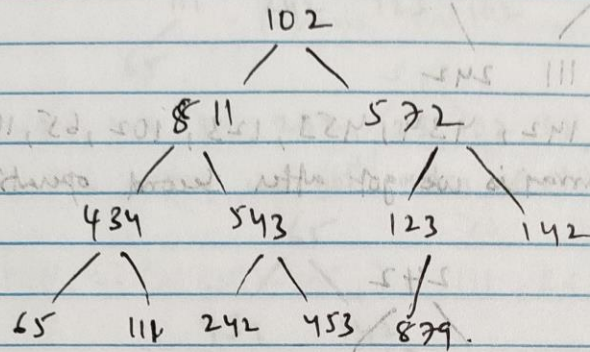
→ So, for max heap, each child node needs to be smaller than the parent node.



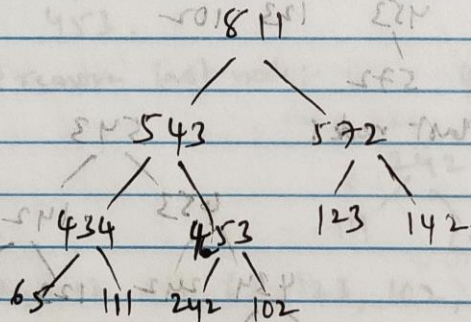
So, the max heap list of array is as given below

879	811	572	434	543	123	142	65	111	242	453	102
-----	-----	-----	-----	-----	-----	-----	----	-----	-----	-----	-----

→ As we have the "max-heap" array here, we have to perform the ~~build~~ heapify operation to the root node by replacing it with the last element and removing it & placing that array to the last (end) of the list.



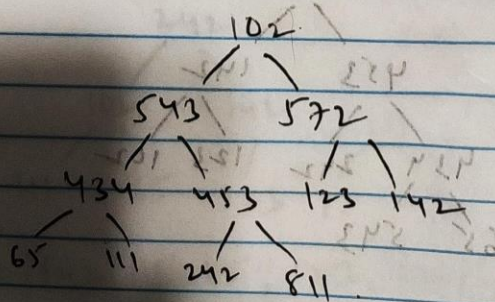
→ As the root node is smaller than child, we need to heapify that until it satisfies the root-child property.



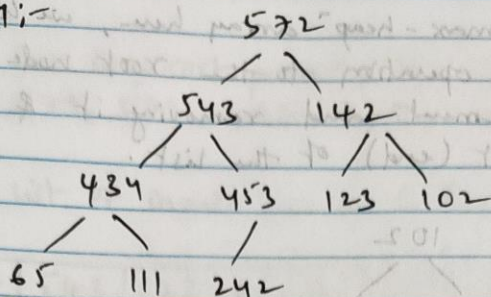
811, 543, 572, 434, 453, 123, 142, 65, 111, 242, 102, 879.

→ This is the array after removing root node and performing heapify operation.

→ continue this process till the array get sorted in ascending order.

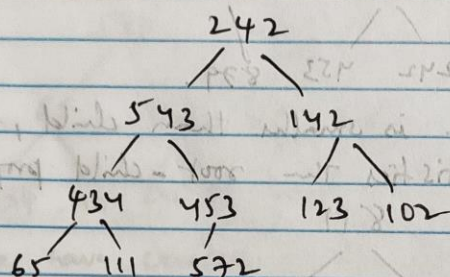


→ heapify:-

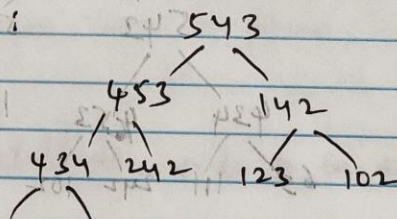


572, 543, 142, 434, 453, 123, 102, 65, 111, 242, 811, 879.

→ The above array is we got after second operation.

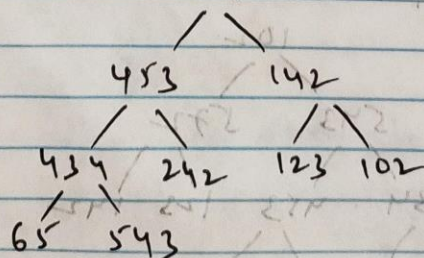


→ heapify & remove last node:

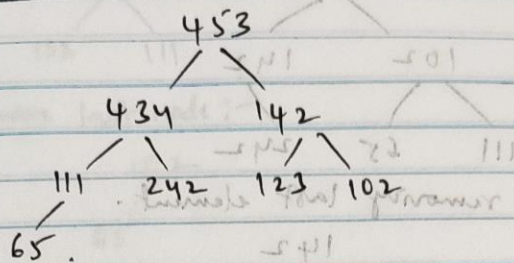


543, 453, 142, 434, 242, 123, 102, 65, 111, 572, 811, 879.

→ Above array is what we got after operation 3

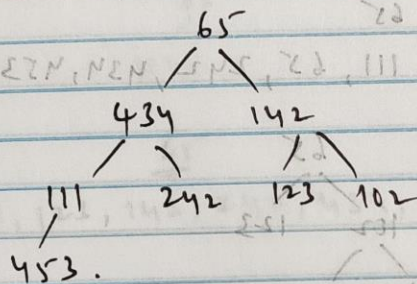


→ heapify & remove last node:-

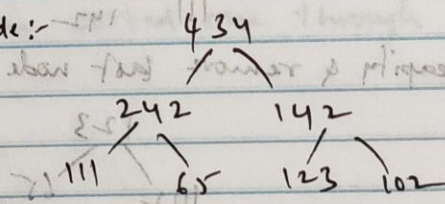


453, 434, 142, 111, 242, 123, 102, 65, 543, 572, 811, 879.

→ This is the array we got after sorting & performing opn 4.

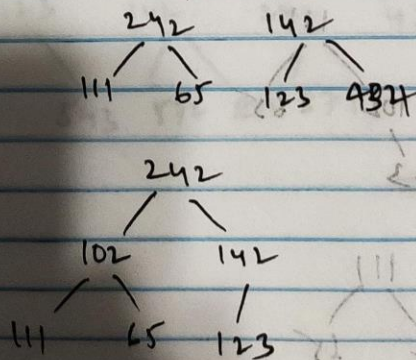


→ heapify & remove last node:-

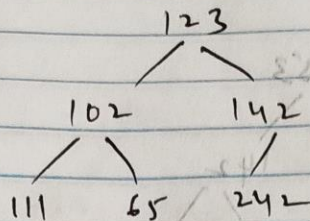


434, 242, 142, 111, 65, 123, 102, 453, 543, 572, 811, 879.

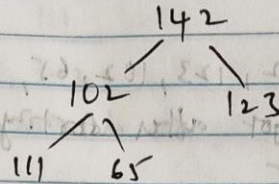
→ heapify:-



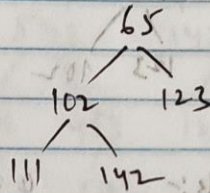
242, 102, 142, 111, 65, 123, 434, 453, 543, 572, 811, 879.



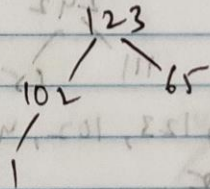
→ heapify & removing last element.



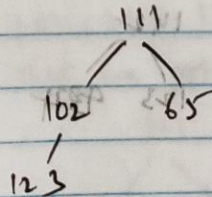
→ 142, 102, 123, 111, 65, 242, 434, 453, 543, 572, 811, 879.



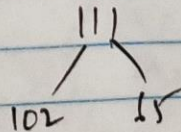
→ heapify & remove last node:



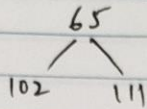
123, 102, 65, 111, 142, 242, 434, 453, 543, 572, 811, 879.



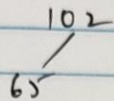
→ heapify:-



111, 102, 65, 123, 142, 242, 434, 453, 543, 572, 811, 879.



→ heapify & remove last node :-



102, 65, 111, 123, 142, 242, 434, 453, 543, 572, 811, 879.



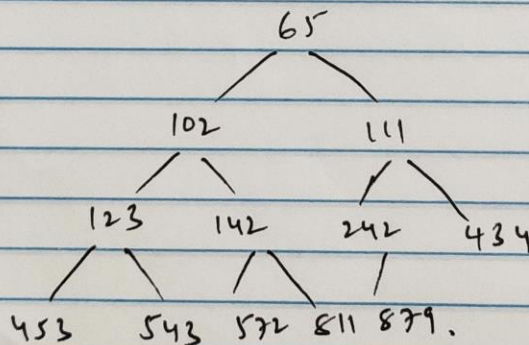
→ heapify :-



65, 102, 111, 123, 142, 242, 434, 453, 543, 572, 811, 879.

→ The above array is the sorted one through heapify opn.

→ The final heap is :-



#

7.15) Sort 3, 1, 4, 1, 5, 9, 2, 6 using Merge sort.

Ans: -

7.15) Sort 3, 1, 4, 1, 5, 9, 2, 6 using Merge Sort.

Ans: → Given array

3	1	4	1	5	9	2	6
---	---	---	---	---	---	---	---

From here, we need to continuously break this array into equal halves, until they get into single items.

3	1	4	1
---	---	---	---

5	9	2	6
---	---	---	---

3	1
---	---

4	1
---	---

5	9
---	---

2	6
---	---

3	1
---	---

4	1
---	---

5	9
---	---

2	6
---	---

$1 < 3$ ↓ $1 < 4$ ↓ $5 < 9$ ↓ $2 < 6$ ↓

1	3
---	---

1	4
---	---

5	9
---	---

2	6
---	---

→ here, we have again merged the items the way they got divided.

→ Need to perform this merging operation by sorting values until one single array forms.

1	3
---	---

1	4
---	---

5	9
---	---

2	6
---	---

$1 < 3$ ↓ $3 < 4$ ↓ $5 < 9$ ↓ $2 < 5 < 6 < 9$ ↓

1	1	3	4
---	---	---	---

2	5	6	9
---	---	---	---

→ merging the left and right array

1	1	3	4
---	---	---	---

2	5	6	9
---	---	---	---

$2 < 3, 4$

1	1	2	3	4	5	6	9
---	---	---	---	---	---	---	---

$\therefore$

1	1	2	3	4	5	6	9
---	---	---	---	---	---	---	---

∴ The above array is the sorted one after performing Merge sort operation.

7.19) Sort 3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5 using quicksort with median-of-three partitioning and a cutoff of 3.

7.19) Sort 3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5 using quicksort with median-of-three partitioning and a cut-off of 3.

Ans:-

To sort the numbers, 3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5 using quick sort with median-of-three partitioning the below are the steps:-

i) choosing a pivot element:- To apply median-of-three partitioning we need to choose the pivot as the median of first, middle and last elements of the array. In this case, the first element is '3', middle element is '5' and last element is '5'.

→ The median of these three values is '5' so we use '5' as the pivot.

ii) partition the array:- Need to re-arrange the values such a way that all the values less than pivot comes before it and greater values comes after it.

→ starting with pointers 'i' & 'j', at the beginning and end of the array respectively.

→ Move 'i' towards right until the element greater than equal to pivot (5) is found.

→ move 'j' towards left until the element less than or equal to pivot (5) is found.

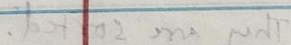
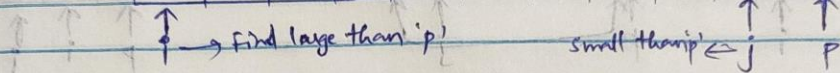
→ If 'i' is still less than or equal to 'j' then swap the elements positions & continue the process.

→ Repeat the process until 'i' becomes greater than 'j'.

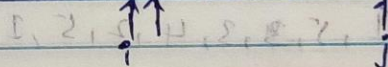
→ Finally, swap the pivot element (5) with the element at position "i-1"



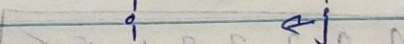
→ First we need to move the pivot to the end of array and start the swapping.



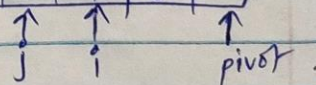
→ So, here we reached the pivot from 'i' so, smaller items from right- needs to insert left of pivot.



→ swap '5' before pivot.



→ '2' being dragged to left



→ So, here i and j are crossed each other so, ~~the~~ no swap is performed. Final part is swapping pivot with element pointed by i .

3 | 1 | 4 | 1 | 3 | 2 | 5 | 5 | 5 | 6 | 9

→ Here, we need to take left half which is from pivot and sort them out by same process. & for larger halves also same.

3 | 1 | 4 | 1 | 3 | 2 | 5 | 5

5 | 6 | 9

v_3

3 | 1 | 4 | 5 | 3 | 2 | 5 | 1

5 | 6 | 9

They are sorted.

1 | 4 | 5 | 3 | 3 | 2 | 5 | 1

1 | 4 | 5 | 3 | 3 | 2 | 5 | 1

1 | 5 | 3 | 3 | 4 | 2 | 5 | 1

1 | 1 | 3 | 3 | 4 | 2 | 5 | 5

1 | 1 | 2 | 3 | 3 | 4 | 5 | 5

→ Merging those two:

1 | 1 | 2 | 3 | 3 | 4 | 5 | 5 | 5 | 6 | 9

#

7.20) Using the quicksort implementation in this chapter, determine the running time of quicksort for

a. sorted input

b. reverse-ordered input

c. random input

Ans: -

A) Sorted input:

If the input data is already sorted in ascending order, quicksort will have a worst-case running time. This is because when the pivot is chosen as the first or last element, the partitioning will result in one subarray with all the elements and the other subarray empty. This creates an unbalanced partition, leading to a running time of $O(n^2)$. However, if the pivot is chosen randomly or using median-of-three partitioning, the running time can be improved to $O(n \log n)$.

B) reverse-ordered input:

If the input data is sorted in descending order (reverse-ordered), quicksort will also have a worst-case running time of $O(n^2)$ if the pivot is chosen as the first or last element. This is because partitioning will result in an unbalanced partition with one subarray containing all the elements and the other subarray empty. However, with a random or median-of-three pivot selection, the running time can be improved to $O(n \log n)$.

C) Random input:

For randomly shuffled or randomly ordered input, quicksort has an average-case running time of $O(n \log n)$. This is because the partitioning step divides the array into two approximately equal-sized subarrays, reducing the problem size by half at each recursive step. With each partition, the array is divided into smaller parts until all elements are sorted.

It's important to note that the worst-case scenario for quicksort can be mitigated by using randomized pivot selection or median-of-three partitioning, which helps achieve a more balanced partition and improves the running time to $O(n \log n)$.

The implementation code for this part is attached in another file.

The below given are the input and output values and running time of the sorted, reverse-ordered and random input.

For sorted input: 2,4,7,9,11,23,34,45,75,87,98

Output: 2,4,7,9,11,23,34,45,75,87,98

Running Time: 8.208E-6 seconds

For reverse-ordered input: 98,87,75,45,34,23,11,9,7, 4, 2

Output: 2,4,7, 9,11,23,34, 45,75,87,98

Running Time: 9.542E-6 seconds

For Random input: 23,45,67,24,62,49,42,89,34

Output: 23,24,34,42,45,49,62,67,89

Running Time: 7.625E-6 seconds